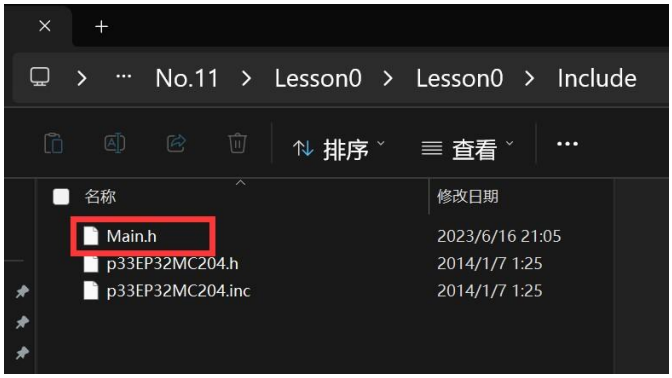
19-单片机 1

大家晚上好，欢迎参加张飞实战电子 FOC 第 11 期学习。上节课我带大家新建工程，开始走了一小步，还没有说完。单片机和后续部分，大家应尽量超前晚课学习，带着问题来听课，课后一定要动手，不然的话，就会有人陆续掉队了。

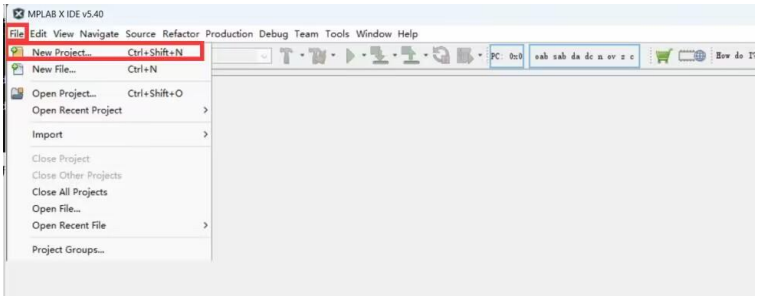
上次复制了官方的三个文件，分别是后缀为.gld .h .inc。官方的文件复制好了，下面我们就需要添加我们自己的文件了。最初始的话，就包含着启动文件，以及大家最熟悉的 Mian.c 文件。如果我们使用官方启动文件的话，则可以直接从 Main.c 开始。启动文件呢，是这样的，如果说我们不使用官方的启动文件，则需要自己编写启动文件。



大家看，如果自己写启动文件的话，需要新建这两个文件。启动文件的话，一般是使用汇编语言来编写的，所以呢，它的文件后缀是.s，然后，跟 Main.c 成对出现的是 Main.h。

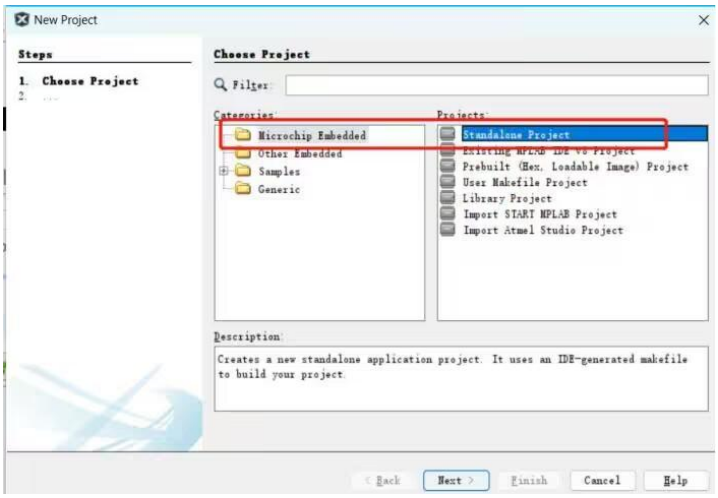


好，这是我们新建工程前的准备工作。大致先建立这些文件啊，后续根据需要再来添加。下面呢，我们要回到 IDE 软件中去，开始新建工程

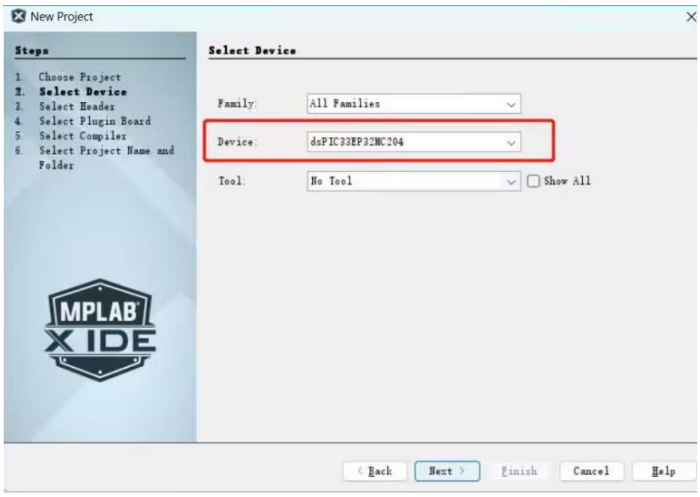


这里，我们点击新建工程

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

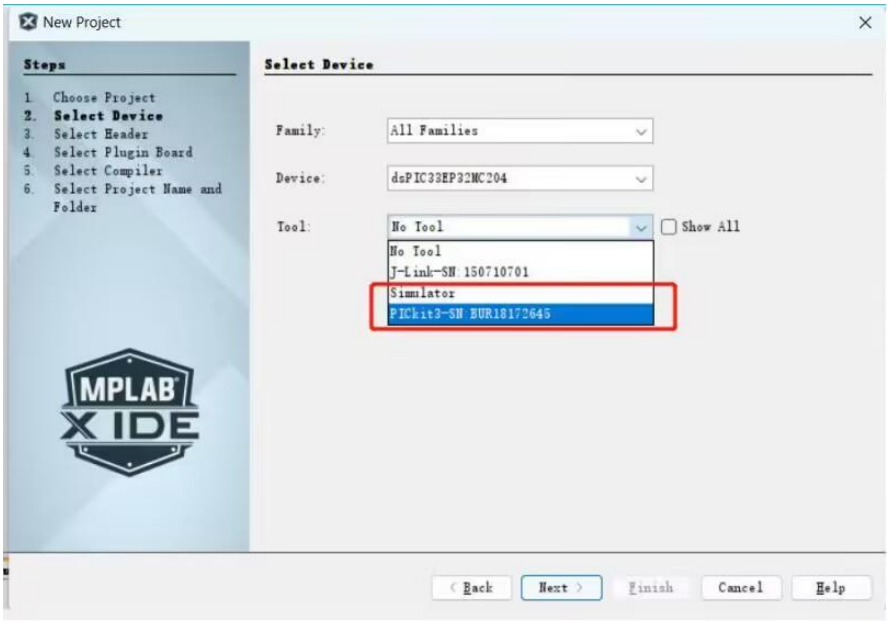


然后呢，我们要选择新建一个空白独立的工程。



接着呢，选择一下对应的芯片型号。

这个地方，可以直接输入，然后，Tool 的话，就是后续下载代码的工具。或者，也可以选择软件仿真工程。



大家看，如果插上仿真器的话，它会识别出来。然后呢，我们可以直接选择对应的工具这里就直接 USB 接到电脑上，就可以识别了，不需要跟开发板连接的。

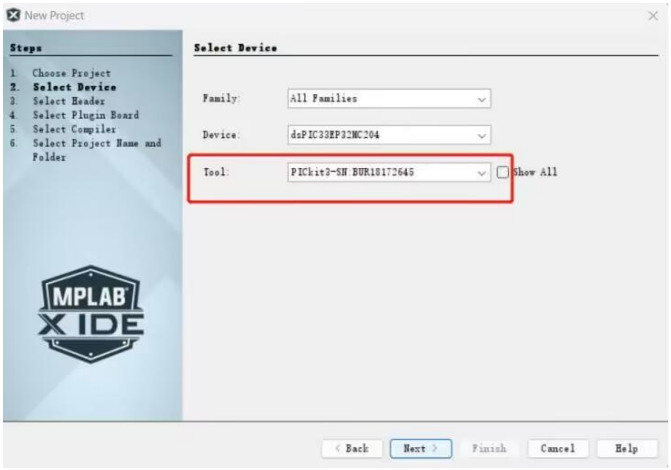
本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

问：老师仿真器是啥呢？

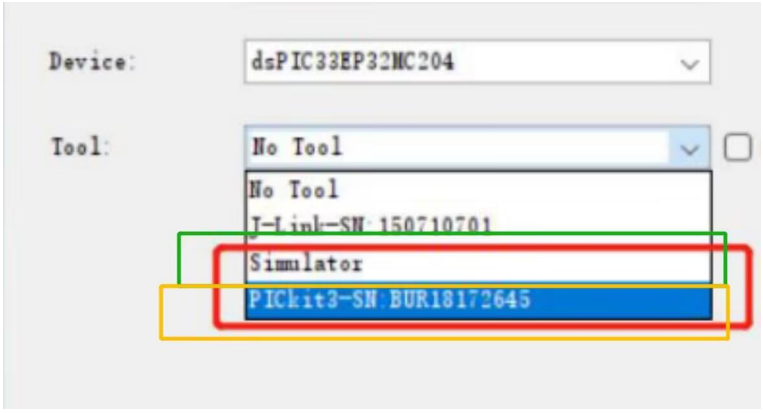


答：就是你收到的红色的小盒子。

问：JLINK　SWD 可以用吗？
答：不可以的。



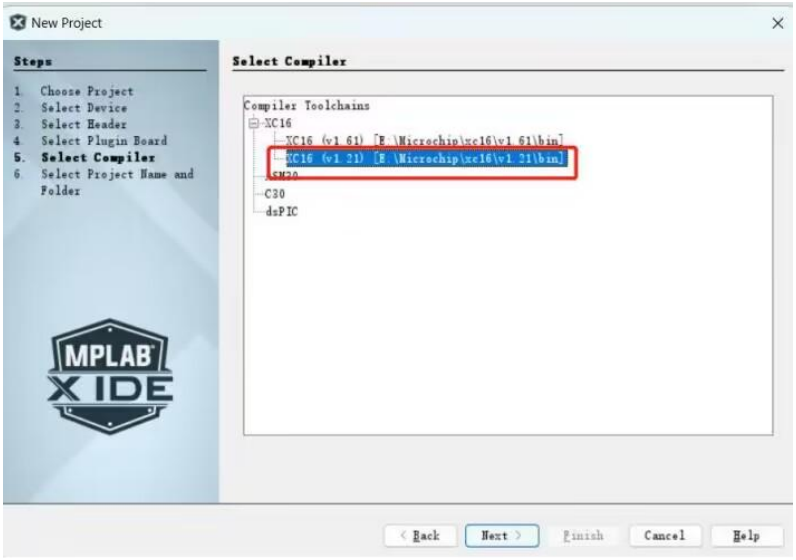
我们后面呢，要使用硬件仿真，所以需要选择对应的仿真工具。当然了，工程新建好了之后，后续也是可以修改的。只要确保对应上就行。



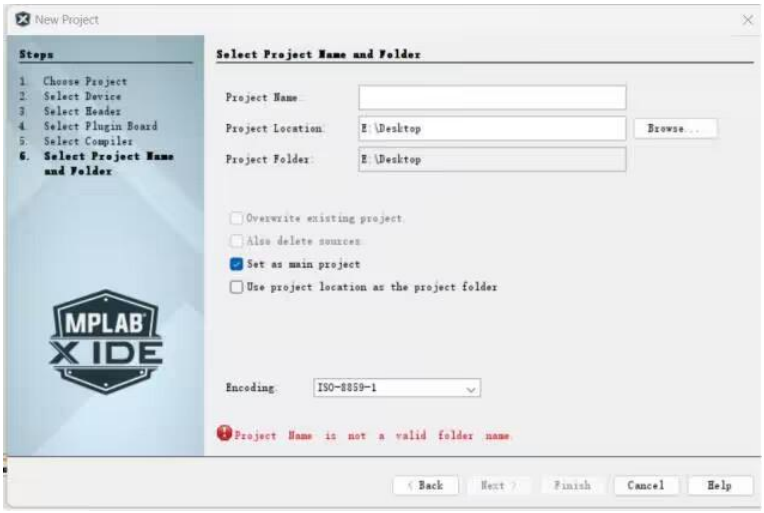
这里选择绿色的就是软件仿真，选择黄色的就是硬件仿真。 硬件仿真需要插上带 USB 接口的硬件仿真器，同时还需要连接目标板。软件仿真外部不需要插仿真器。

问：这个仿真有什么功能，能看哪些东西呢？
答：这个需要你自己动手去看，没有单片机基础的，还需要先学习单片机课程。

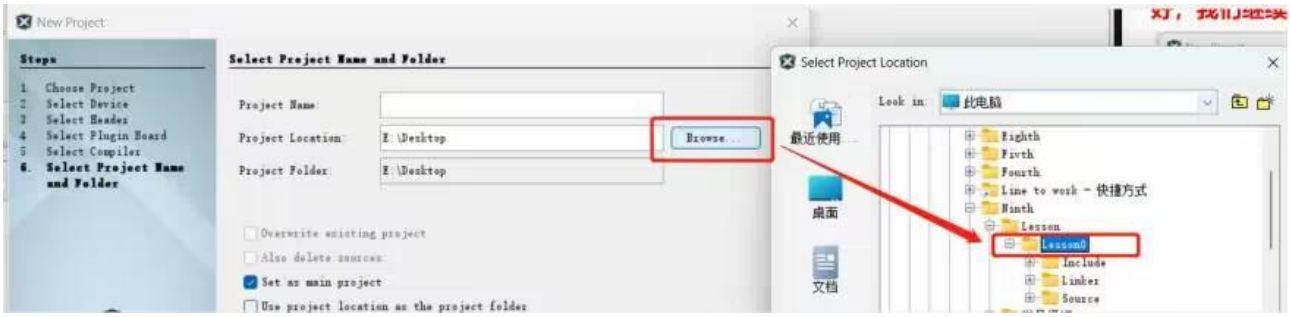
本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



接着呢，我们需要选择编译工具。我们课上都约定使用 V1.21 版本的编译器，所以呢需要选择对应选项啊。如果说这个地方没有的话，就是编译器软件没有安装或者没有添加。没有安装 XC16 V1.21 的学员需要课后安装一下。 V1.61 会有部分内容和我们的代码不兼容。

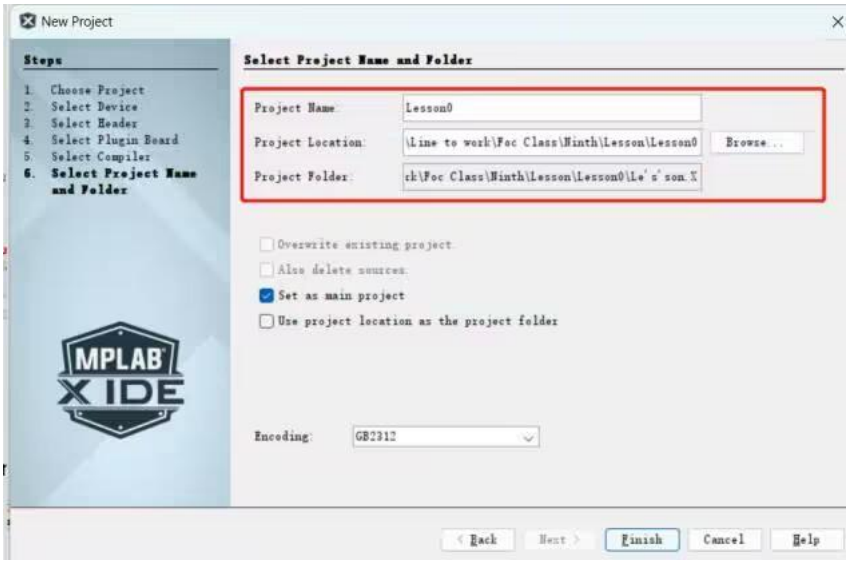


然后，会跳转到这个界面，

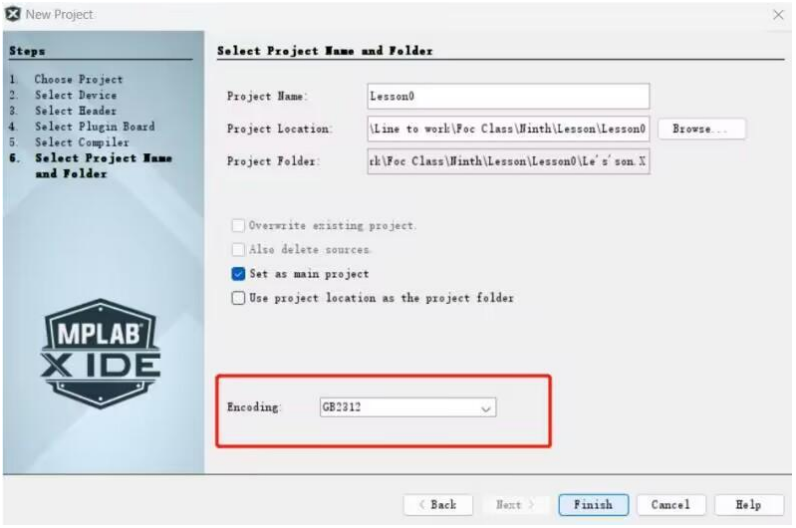


然后呢，我们需要选择工程存放的路径。这里呢，就选择到我们建立的 Lesson0 这个目录下即可。

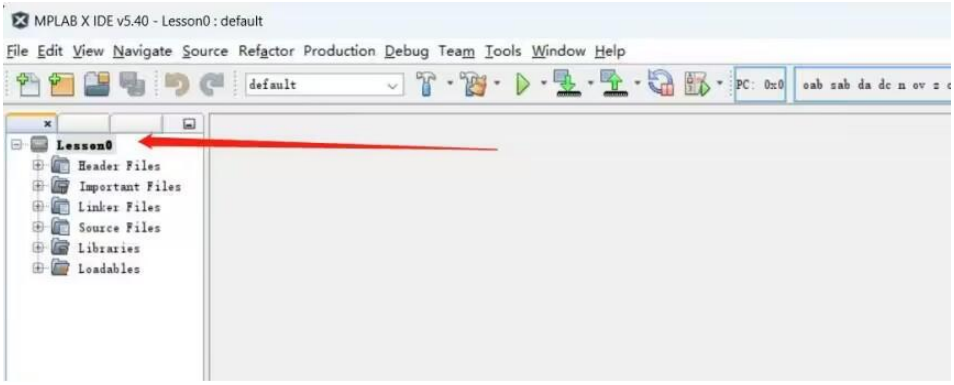
本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



选择好之后，给工程起一个名字。这里，我也叫 Lesson0 了。

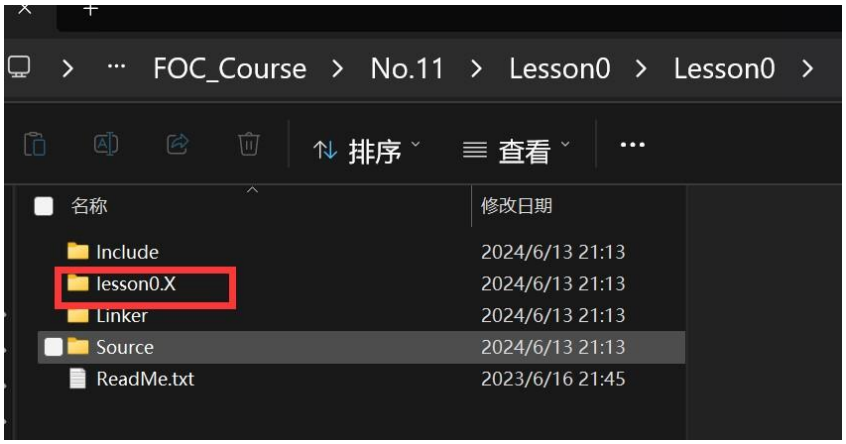


然后，这个地方的编码格式选择 GB2312。主要是要支持中文显示。也就是方便后续的中文注释。都选择好之后，点击 Finish。



此时呢，软件中就出现了对应的工程。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

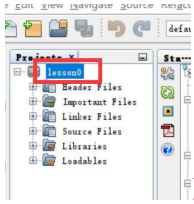


然后呢，大家会看到工程目录下，出现了.X 的文件。这个呢，就是对应的工程文件了。那么，到这里我们就成功把一个工程新建好了。

问：仿真器没接可以吗？
答：使用软件仿真可以不接硬件仿真器，使用硬件仿真需要接硬件仿真器。

我们平时打开工程的时候，也是打开这个.X 文件就可以了。到这一步大家一定要能自己实现啊，后面都是一环套一环，大家一定要跟上啊。

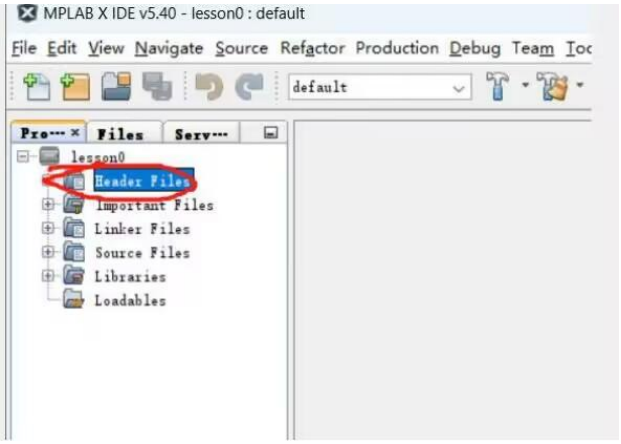
问：选了 No tool 后面还可以改吗？



答：可以的。lesson0 右键属性里修改。

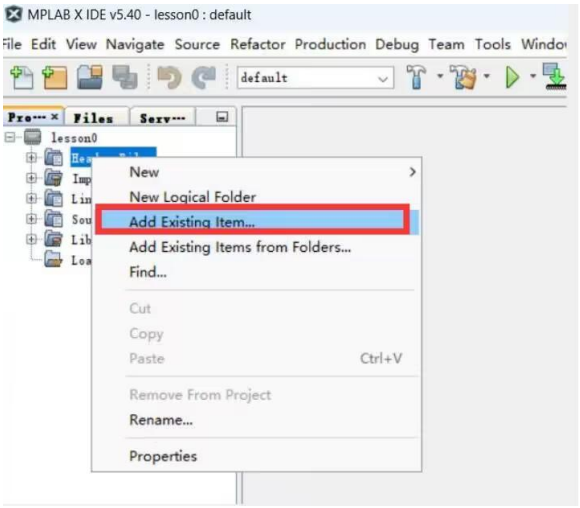
问：这个软件仿真有一些外设功能是实现不了的吧，例如采样？
答：软件仿真实现不了，硬件仿真可以。

好的，接着呢，就开始往工程中添加文件

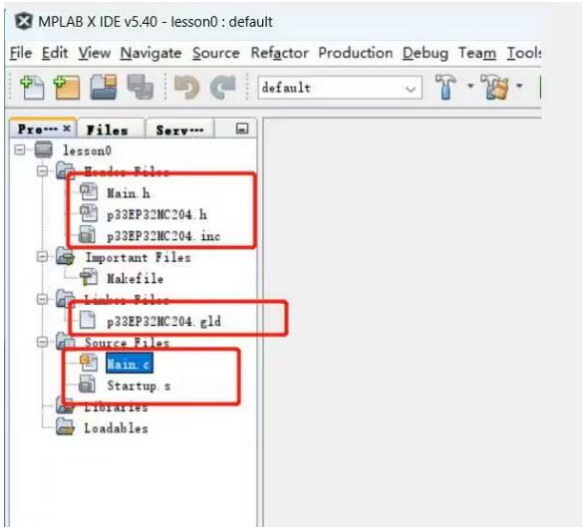


例如，我们要添加头文件。就右键点击这个目录。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

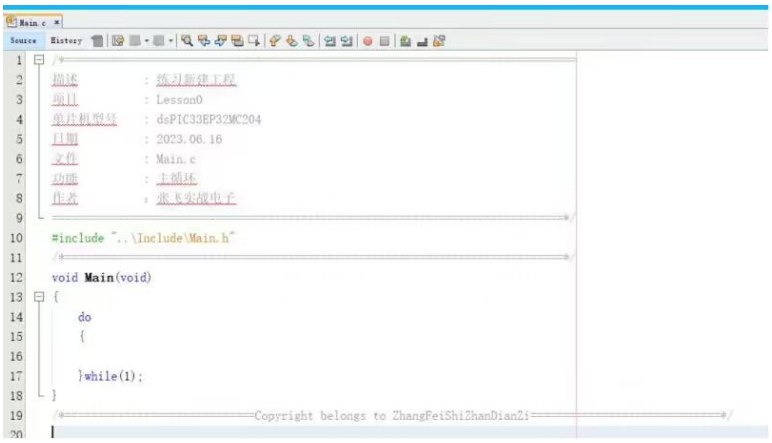


然后，选择这里的添加外部文件



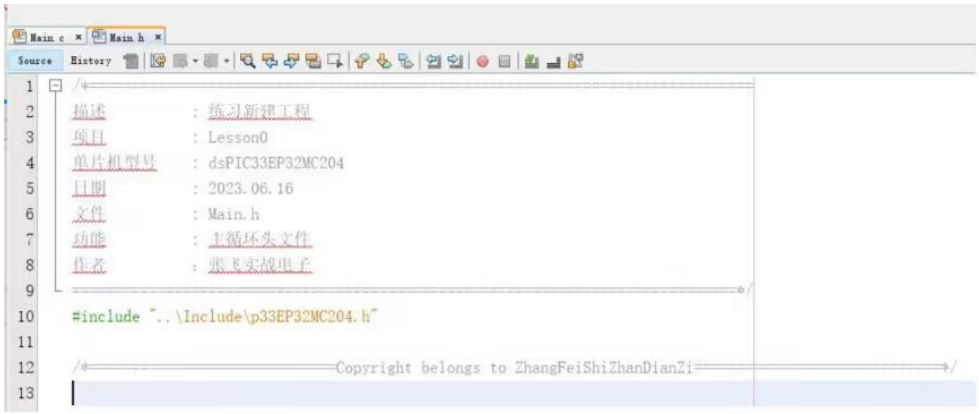
按照一样的方法，把我们之前新建的文件都添加进来。就是上图所示这样了，大家操作一下，看看有没有问题。

好，接着往 Main.c 中添加内容



对应的内容呢，大致可以写为这样。

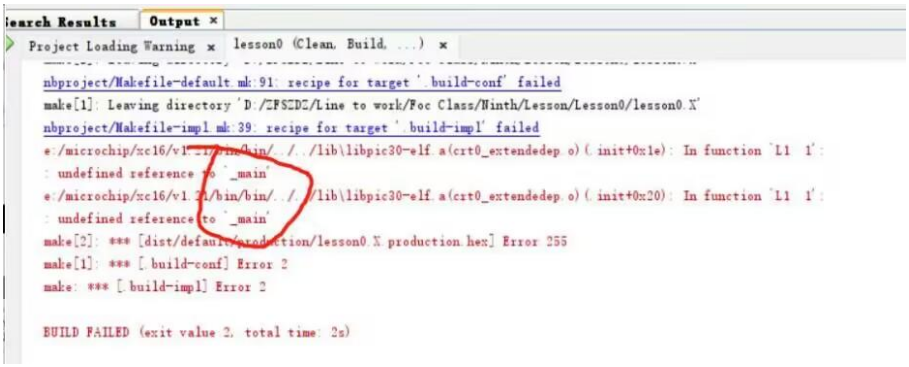
本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



对应的头文件呢，也可以写为这样。头文件中，引用官方的头文件。此时，如果我们要编译的话，会报错。

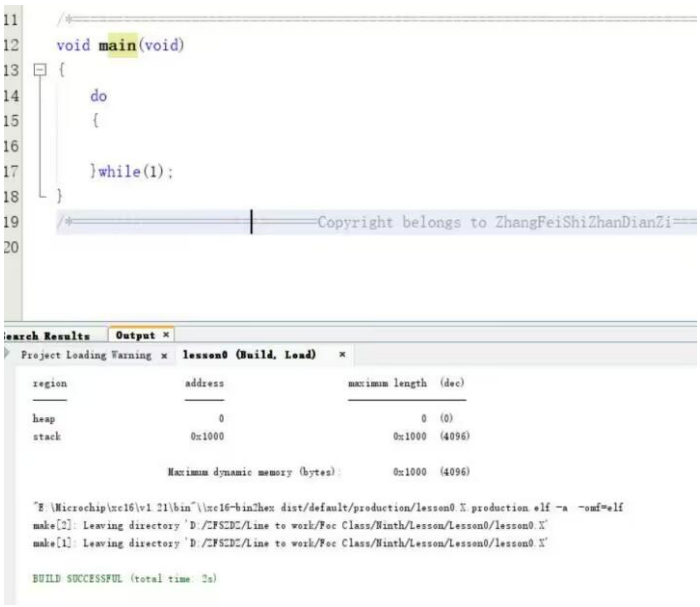


红圈中的图标跟后面一个的区别是　这个按钮只重新编译修改的部分，而后面的按钮要全部重新编译。

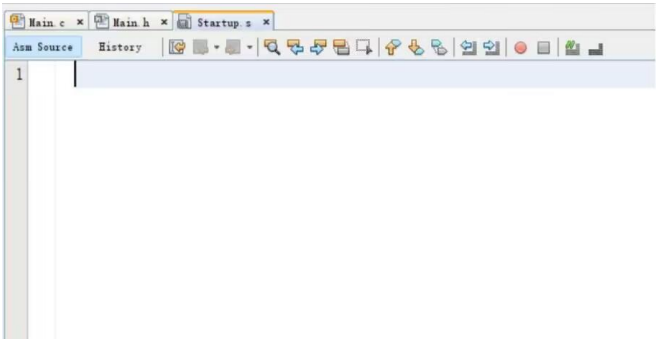


编译之后，会提示主函数不对。这个原因呢，是因为使用官方的启动文件，会默认要 main 这个标识。而我们是　Main 的话，它就不识别了。

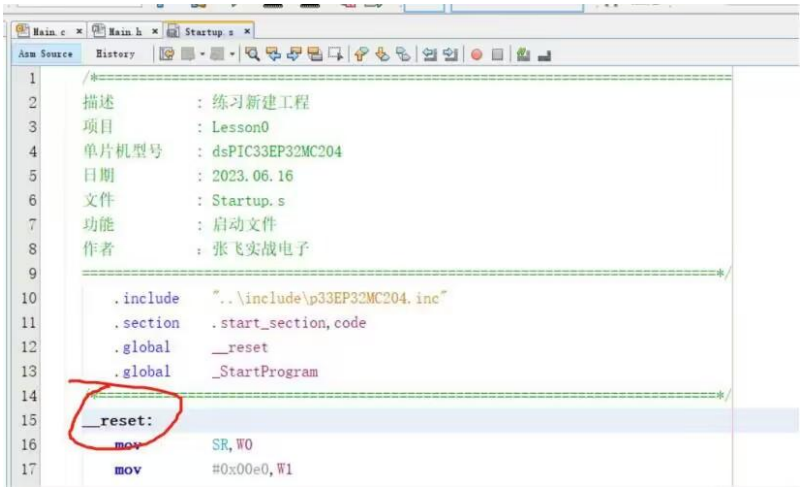
本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



如果是这样的话，修改一下，编译就完成了。暂且，我们先用小写的 main 函数标识了我们这里没有写启动文件的话就是默认使用官方的 。



这个文件是空的，所以不会使用这个启动文件。此时编译的话，还是使用官方的哈
也即是说，刚才这个步骤，不加入我们的启动文件，就是使用官方的启动文件来进行开发就可以直接在 main 函数中来编写需要的功能了。如果我们要写启动文件的话，就需要自己添加对应的内容。



大家看，如果要使用自己写的启动文件的话，就需要添加上__reset 复位函数。也就是说，是否使用我们自己的启动文件，就是看是否我们自己加上__reset 这个函数。如果我们加了__reset，就会优先使用我们的启动文件。如果我们没有加，就默认使用官方的启动文件。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

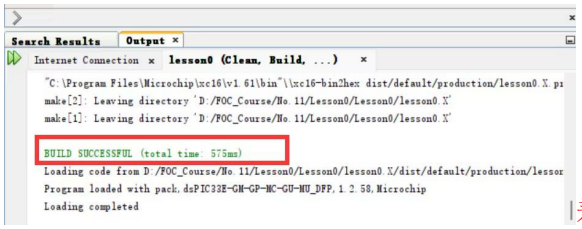
问：官方的启动文件能看到吗？
答：看不到。

问：那官方启动文件也不要加载在我们的文件夹里么？
答：不需要加载。

问：include 后面直接写..就行，没有包含文件路径这和说法吧？
答：include 后面的 .. 表示当前.c 文件的上一级目录

问：include 这个斜杠和老师的不一样，不影响吗？
答：这个有讲究，要使用 \

问：是不是编译只要不出现红色就通过了？

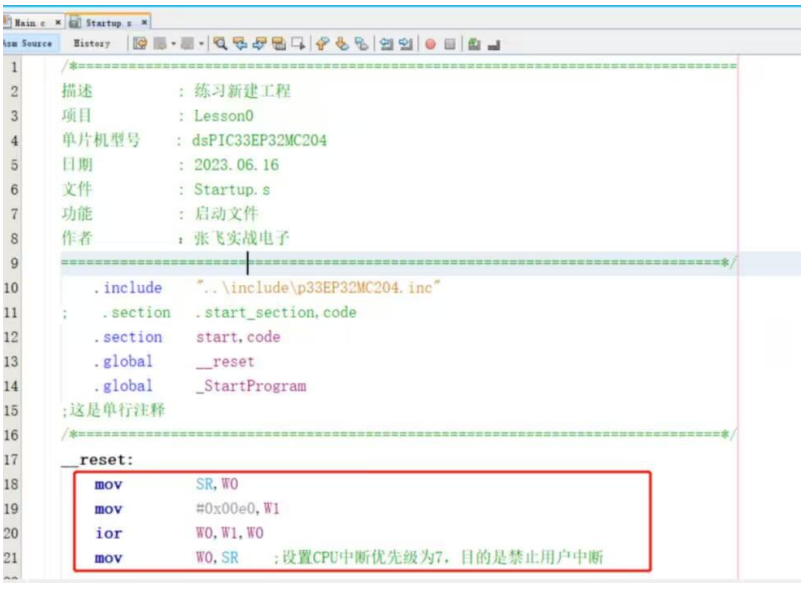


答：看到 BUILD SUCCESSFUL 就代表编译成功。

关于单片机方面有很多资料在群文件中。我们后面内容，要讲一下启动文件，对于单片机的功能会逐渐深入，所以关于单片机的资料，大家课后需要自己学习，带着问题来上课比较好，资料都在群文件中。一方面是单片机的数据手册，需要了解单片机的主要架构和内核特点。另一方面，需要阅读程序员参考手册，汇编和 C 语言部分。有问题和不理解的需要多沟通交流。

问：启动文件直接用官方的能够支撑整个课程的软件吗？
答：不能。

今晚，我们先来一小段内容，相关内容大家先体会一下



本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

这是启动文件开始一小段，也是单片机复位开始的地方。上面的 include section global 大家都理解么？

include 是包含头文件，就是说编译器编译当前文件的时候，会把后面的.inc 文件包含进来。section 是指段， start 是这个段的名字，code 是指这个段是代码段。

global 是声明后面的标号或变量或函数为全局类型。

第一句 mov SR,W0 是什么意思，知道的说说看。

对的，mov SR,W0 就是将 SR 寄存器中的值传送到寄存器 W0 中。

第二句，mov #0x00e0,W1 是把立即数 0x00e0 传送至 W1 寄存器 。

ior W0,W1,W0 又是什么意思呢？

IOR Wb 和 Ws 逻辑或

实现于：	PIC24F	PIC24H	PIC24E	dsPIC30F	dsPIC33F	dsPIC33E
	X	X	X	X	X	X

语法：	{标号 :}	IOR{.B}	Wb,	Ws, [Ws], [Ws++], [Ws--], [++Ws], [--Ws],	Wd [Wd] [Wd++] [Wd--] [++Wd] [--Wd]
操作数：	Wb ∈ [W0 ... W15] Ws ∈ [W0 ... W15] Wd ∈ [W0 ... W15]				
操作：	(Wb).IOR.(Ws) → Wd				
受影响的状态位：	N 和 Z				

指令编码：	0111	0www	wBqq	qddd	dppp	ssss

在程序员手册中有对汇编指令的解释，以上三句连起来，要表达什么意思呢？

```
1  /*=====================================================================
2  描述          : 练习新建工程
3  项目          : Lesson0
4  单片机型号    : dsPIC33EP32MC204
5  日期          : 2023.06.16
6  文件          : Startup.s
7  功能          : 启动文件
8  作者          : 张飞实战电子
9  =====================================================================*/
10     .include    "..\include\p33EP32MC204.inc"
11 ;    .section    .start_section,code
12     .section    start,code
13     .global     __reset
14     .global     _StartProgram
15 ;这是单行注释
16 /*=====================================================================*/
17 __reset:
18     mov         SR,W0
19     mov         #0x00e0,W1
20     ior         W0,W1,W0
21     mov         W0,SR       ;设置CPU中断优先级为7，目的是禁止用户中断
```

前三句执行完之后，是不是要将 W0 的 bit5,bit6,bit7 置 1 啊？

第四句 mov W0,SR，将 W0 寄存器中的值送回 SR。

这四句话，最终实现的是将 SR 寄存器的 bit5 bit6 bit7 位置 1。

我们来看一下 SR 寄存器这几位代表什么，是什么意思？

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

寄存器 3-1：　SR：CPU 状态寄存器

R/W-0	R/W-0	R/W-0	R/W-0	R/C-0	R/C-0	R-0	R/W-0
OA(1)	OB(1)	SA(1,4)	SB(1,4)	OAB(1)	SAB(1)	DA(1)	DC
bit 15							bit 8

R/W-0(2,3)	R/W-0(2,3)	R/W-0(2,3)	R-0	R/W-0	R/W-0	R/W-0	R/W-0
	IPL<2:0>		RA	N	OV	Z	C
bit 7							bit 0

图注：	C = 可清零位		
R = 可读位	W = 可写位	U = 未实现位，读为 0	
-n = POR 时的值	1 = 置 1	0 = 清零	x = 未知

bit 15　**OA：** 累加器 A 溢出状态位 (1)
1 = 累加器 A 已溢出
0 = 累加器 A 未溢出

bit 14　**OB：** 累加器 B 溢出状态位 (1)
1 = 累加器 B 已溢出
0 = 累加器 B 未溢出

bit 13　**SA：** 累加器 A 饱和 “粘住” 状态位 (1,4)
1 = 累加器 A 饱和或在某时已经饱和
0 = 累加器 A 未饱和

bit 12　**SB：** 累加器 B 饱和 “粘住” 状态位 (1,4)
1 = 累加器 B 饱和或在某时已经饱和
0 = 累加器 B 未饱和

bit 11　**OAB：** OA 和 OB 组合的累加器溢出状态位 (1)
1 = 累加器 A 或 B 已溢出
0 = 累加器 A 和 B 都未溢出

bit 10　**SAB：** SA 和 SB 组合的累加器 “粘住” 状态位 (1)
1 = 累加器 A 或 B 饱和或在过去某时已经饱和
0 = 累加器 A 和 B 都未饱和

bit 9　**DA：** DO 循环活动位 (1)
1 = 正在进行 DO 循环
0 = 不在进行 DO 循环

bit 8　**DC：** MCU ALU 半进位 / 借位标志位
1 = 结果的第 4 个低位（对于字节大小的数据）或第 8 个低位（对于字大小的数据）发生了进位
0 = 结果的第 4 个低位（对于字节大小的数据）或第 8 个低位（对于字大小的数据）未发生进位

注　**1：** 该位仅在 dsPIC33EPXXXMC20X/50X 和 dsPIC33EPXXXGP50X 器件上可用。
2： IPL<2:0> 位与 IPL<3> 位（CORCON<3>）组合形成 CPU 中断优先级。如果 IPL<3> = 1，那么括号中的值表示 IPL。当 IPL<3> = 1 时，禁止用户中断。
3： 当 NSTDIS 位（INTCON1<15>）= 1 时，IPL<2:0> 状态位是只读的。
4： 对 SR 寄存器的数据写操作可以修改 SA 和 SB 位，方法是向 SA 和 SB 写入数据或清零 SAB 位。要避免可能出现的 SA 或 SB 位写竞争条件，不要使用位操作来修改 SA 和 SB 位。

寄存器 3-1：　SR：CPU 状态寄存器（续）

bit 7-5　**IPL<2:0>：** CPU 中断优先级状态位 (2,3)
111 = CPU 中断优先级为 7（15）；禁止用户中断
110 = CPU 中断优先级为 6（14）
101 = CPU 中断优先级为 5（13）
100 = CPU 中断优先级为 4（12）
011 = CPU 中断优先级为 3（11）
010 = CPU 中断优先级为 2（10）
001 = CPU 中断优先级为 1（9）
000 = CPU 中断优先级为 0（8）

bit 4　**RA：** REPEAT 循环活动位
1 = 正在进行 REPEAT 循环
0 = 不在进行 REPEAT 循环

bit 3　**N：** MCU ALU 负标志位
1 = 结果为负
0 = 结果为非负（零或正值）

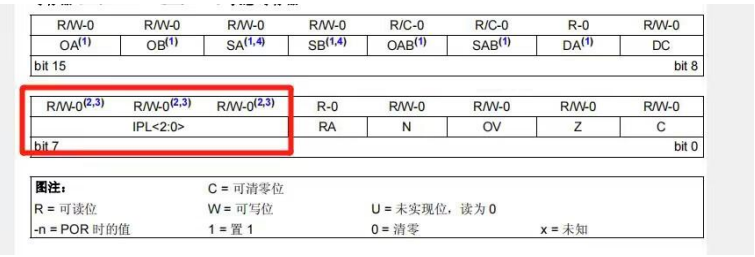
bit 2　**OV：** MCU ALU 溢出标志位
该位用于有符号的算术运算（以二进制补码方式进行）。它表示量值上的溢出，这种溢出将导致符号位改变状态。
1 = 有符号算术运算中发生溢出（本次算术运算）
0 = 未发生溢出

bit 1　**Z：** MCU ALU 全零标志位
1 = 影响 Z 位的任何运算在过去某时已将该位置 1
0 = 影响 Z 位的最近一次运算已将该位清零（即运算结果非零）

bit 0　**C：** MCU ALU 进位 / 借位标志位
1 = 结果的最高有效位发生了进位
0 = 结果的最高有效位未发生进位

注　**1：** 该位仅在 dsPIC33EPXXXMC20X/50X 和 dsPIC33EPXXXGP50X 器件上可用。
2： IPL<2:0> 位与 IPL<3> 位（CORCON<3>）组合形成 CPU 中断优先级。如果 IPL<3> = 1，那么括号中的值表示 IPL。当 IPL<3> = 1 时，禁止用户中断。
3： 当 NSTDIS 位（INTCON1<15>）= 1 时，IPL<2:0> 状态位是只读的。
4： 对 SR 寄存器的数据写操作可以修改 SA 和 SB 位，方法是向 SA 和 SB 写入数据或清零 SAB 位。要避免可能出现的 SA 或 SB 位写竞争条件，不要使用位操作来修改 SA 和 SB 位。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



R/W-0	R/W-0	R/W-0	R/W-0	R/C-0	R/C-0	R-0	R/W-0
OA(1)	OB(1)	SA(1,4)	SB(1,4)	OAB(1)	SAB(1)	DA(1)	DC
bit 15							bit 8

R/W-0(2,3)	R/W-0(2,3)	R/W-0(2,3)	R-0	R/W-0	R/W-0	R/W-0	R/W-0
IPL<2:0>			RA	N	OV	Z	C
bit 7							bit 0

图注：	C = 可清零位		
R = 可读位	W = 可写位	U = 未实现位，读为 0	
-n = POR 时的值	1 = 置 1	0 = 清零	x = 未知

bit5 bit6 bit7 就对应图中的红框中位置啊



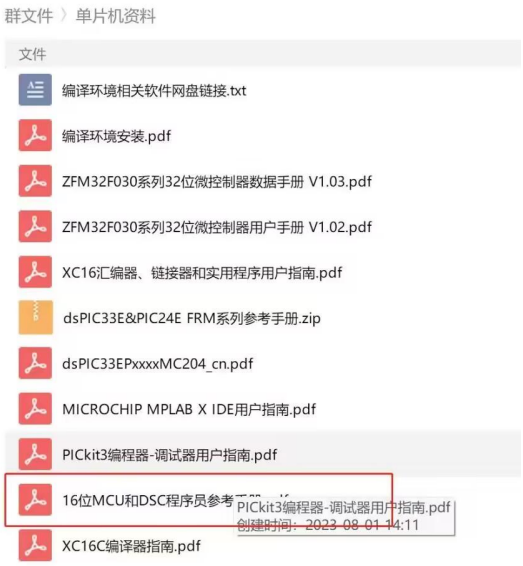
寄存器 3-1： SR：CPU 状态寄存器 （续）

bit 7-5 IPL<2:0>：CPU 中断优先级状态位 (2,3)

111 = CPU 中断优先级为 7（15）：禁止用户中断
110 = CPU 中断优先级为 6（14）
101 = CPU 中断优先级为 5（13）
100 = CPU 中断优先级为 4（12）
011 = CPU 中断优先级为 3（11）
010 = CPU 中断优先级为 2（10）
001 = CPU 中断优先级为 1（9）
000 = CPU 中断优先级为 0（8）

禁止用户中断，用户中断不再响应。

问：mov 是不是前后移动的数据反了？
答：不同的单片机对 MOV 的解释会不一样。



对 MOV 的解释需要看程序员手册。

今天内容我们就到这里，下节继续分析启动文件，大家课后可以先分析啊。谢谢大家，晚安！

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。